

Agentic AI 기반 멀티클라우드 CNAPP 보안 플랫폼

AWS 워크로드 × Azure 신원 — 크로스클라우드 공격경로를 스스로 조사하는 보안 플랫폼

김진우 Kim-Jin-Woo

이준형 Lee-Jun-Hyung

26-06-27 ~ 26-07-31

데모 youtu.be/YaPADbf3t94

GitHub github.com/jun0601/cnapp-agentic

목차

01. 개요

- 기획 배경
- 역할 분담 & 협업 구조
- 전체 아키텍처

02. 탐지 · AI 분석

- Cross-Cloud 공격 시나리오 & 취약 App
- 관제 콘솔 소개
- 수집 · 정규화 파이프라인
- attack-path 상관 엔진
- RAG
- Agentic Engine 4단계
- Bedrock 능동조사
- 관제 콘솔 실 탐지 결과

03. 인프라 · 운영

- 보안 거버넌스 & 플랫폼 자기방어
- 인프라 설계
- FinOps 비용 전략
- CI/CD
- 운영 관측

04. 트러블슈팅

- X-Ray Error 노드 규명
- Prometheus 미기동
- Azure 구독 제약 오픈
- 정규화 파서 크래시

05. 결과

- 성과
- 데모 영상

“기존 클라우드 보안을 더 보완할 방법을 찾다가 **CNAPP**로 향하는 흐름을 발견했다. 다만 **CNAPP**도 findings 우선순위는 매기지만, 그게 진짜 위험인지 조사해서 **판정**하는 건 여전히 사람 몫이었다. 이러한 프로세스를 **AI**가 **스스로** 해낼 수 있다면 어떨까 싶어 **AI**를 핵심 축으로 **CNAPP**를 구현해보고자 프로젝트를 기획하게 되었다. 이 과정에서 마주한 문제들을 아래와 같이 정의하고, 각각에 대한 해결 접근을 직접 설계·구현했다.”

문제 정의



보안 신호 파편화

Security Hub(ASFF) · Prowler(OCSF) · Trivy(trivy-json) 등 스캐너마다 출력 스키마와 심각도 기준이 전부 달라, 담당자가 여러 콘솔을 오가며 수동으로 취합·재판단해야 함



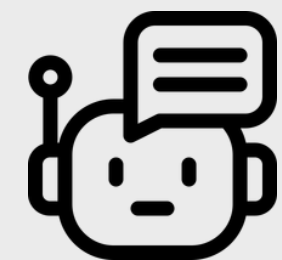
개별 finding(보안 위험 신호)의 함정

Medium 취약점 1건과 Low 등급 취약점 1건을 따로 보면 둘 다 급하지 않아 보이지만, 실제로는 하나의 공격 체인일 수 있음. **finding**을 개별 심각도로만 나열하면 이 연결 관계 자체가 안 보임



클라우드 단일 도구의 사각지대

AWS 전용 도구는 AWS만, Azure 전용 도구는 Azure만 볼 수 있어, **AWS → Azure**로 이어지는 **경계 횡단** 공격의 전체 경로를 파악 못함



대화형 AI의 한계

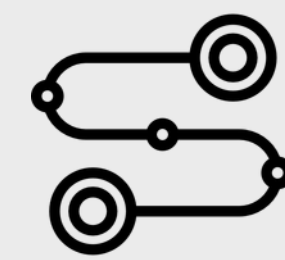
기존 AI 도구 대부분은 이미 수집된 데이터를 요약해 답하는 **챗봇**에 머물기 때문에, 결국 실제 문제를 해결하려면 분석가가 직접 콘솔을 확인해야 함

해결 접근



CNAPP

CSPM · CIEM · Vuln · KSPM · DSPM · **attack-path** 6개 기둥(6 Pillars)을 **OCSF-lite** 공통 스키마로 정규화하여, 스캐너가 몇 개든 같은 체계로 들어와 단일 콘솔·단일 우선순위 기준으로 통합



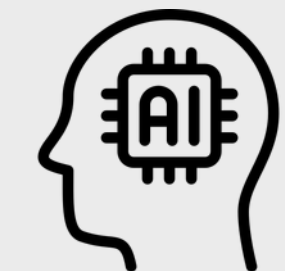
Attack Path

침투 진입점→권한 확대→**자격증명 탈취**→**신원 장악**으로 이어지는 **상관 규칙**으로 finding들을 노드·엣지 그래프로 연결하여, 개별로는 급하지 않던 finding들이 실제로 이어지면 최우선순위로 끌어올림



Cross-Cloud

AWS에 평문으로 남은 **Azure 자격증명**을 발견하면 그 경로를 엣지로 별도 표시해 **그래프상**에서 바로 구분하고, Azure는 **Prowler**로 Entra 테넌트를 스캔해 finding 확보



Agentic AI

Triage→**Hypothesis**→**Evidence**→**Reasoning** 4단계 루프에서, Evidence 단계의 **Bedrock**이 **read-only API**를 **직접** 호출하고, 응답을 근거로 판정함

역할 분담 & 협업 구조

9개 영역을 반씩 나누되, 서로의 영역까지 이해하며 진행 — 계약(Contract) 기반 병렬 개발

역할 분담

이준형

김진우

01 토대 — 앱 · 인프라 · 인증

취약·관제 앱 2종 · CI/CD · Shift-Left · 공유 인프라

앱 · 토대

모니터링·관측 (Grafana · X-Ray)

Cognito 허브 · 콘솔 SSO · 로그인 감사

인증 · SSO

Entra 테넌트 · App Registration · SAML IdP

02 탐지 — 스캐너 · 수집

CSPM (Config · Prowler · Security Hub · Macie)

스캐너 CSPM

워크로드 (Trivy · kube-bench)

IAM Access Analyzer (AWS)

스캐너 CIEM

Entra ID (Azure)

수집부 (EventBridge → SQS)

수집 · 정규화

정규화부 (Lambda → OCSF-lite)

03 분석 · 대응 — AI 파이프라인 · 조치

그래프 데이터 모델

attack-path

상관 로직 (5규칙) · 내러티브

Evidence (tool use) · Triage

엔진 (Bedrock)

Hypothesis · Reasoning · Orchestrator

코퍼스 · 임베딩 · pgvector 적재

RAG

검색 · LLM 답변 생성

조치(Remediation·SFn·감사) · 하드닝(WAF·JWT·TLS)

조치 · 거버넌스

CloudTrail 감사 · 키리스 인증(Entra OIDC)

협업 구조

☑ 계약(Contract) 기반 설계

JSON 스키마 7종 · INTERNAL Control 15종을 SSOT로 먼저
확정 → 인터페이스가 고정되니 서로 안 기다리고 병렬 개발

☑ mock-first → 실 연동

mock 데이터로 각자 개발·검증 후 mock=False 한 줄로 실 클라
우드 전환 (계약이 같아 이음새 안 깨짐)

☑ CI Validate 게이트

계약 위반(스키마 불일치·매핑 오류)을 매 커밋 자동 차단

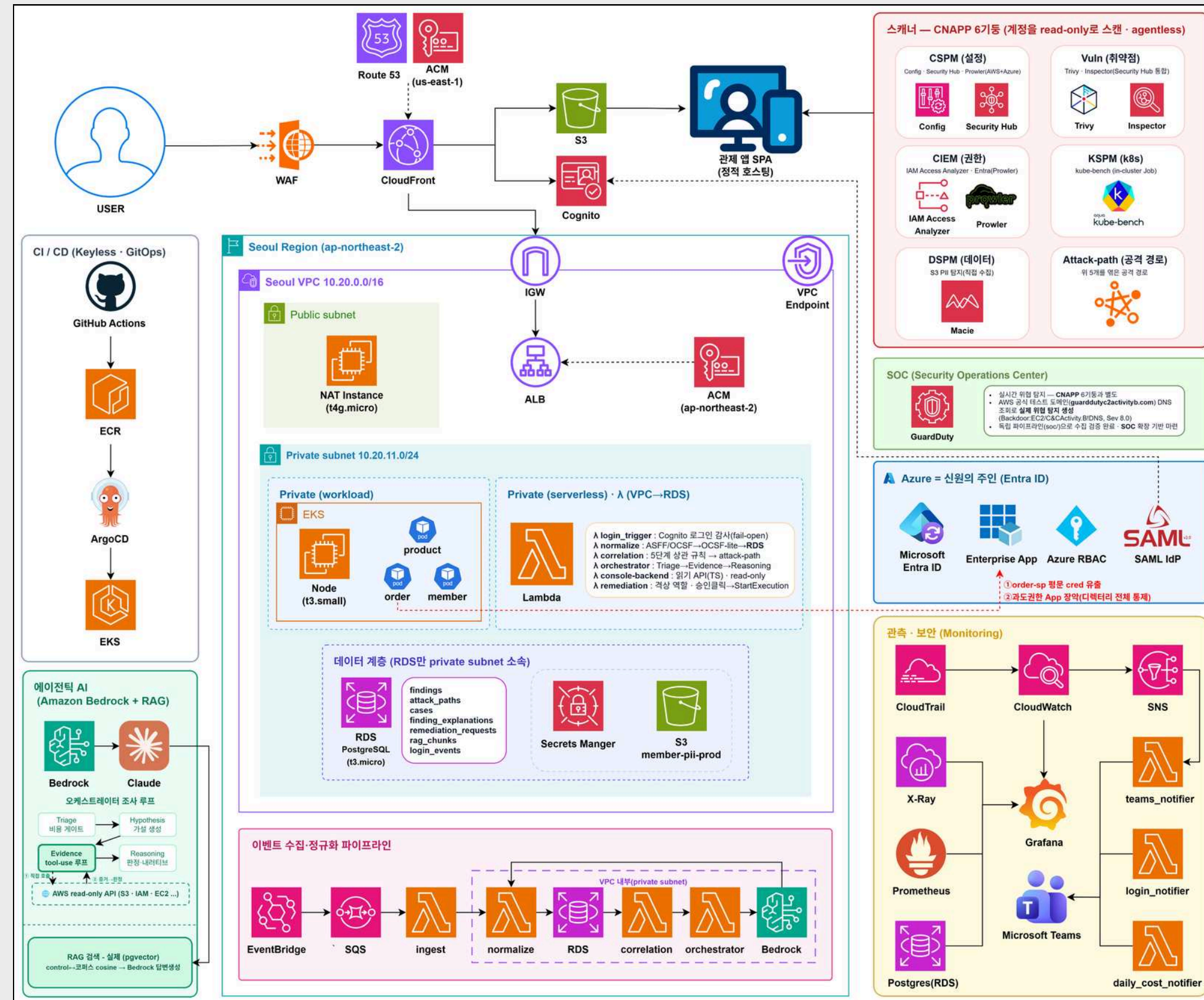
☑ git 규율

작업 전 pull · [PULL 필요] 태그 · 변경 로그로 매 결정 공유

전체 아키텍처

01. 개요

AWS는 워크로드(EKS·컨테이너·데이터)의 주인, **Azure**는 신원(Entra ID)의 주인



☑ CNAPP 정의

- **CSPM** : 클라우드 설정 오류
 - 탐지: Prowler · AWS Config · Security Hub
- **CIEM** : 과도한 IAM·Entra 권한
 - 탐지: IAM Access Analyzer · Prowler (Entra ID)
- **Vuln** : 이미지·코드 CVE
 - 탐지: Trivy · Inspector · ECR 스캔
- **KSPM** : 쿠버네티스 설정 오류
 - 탐지: kube-bench (CIS 벤치마크 기준) · Trivy (k8s)
- **DSPM** : 민감 데이터(PII) 노출
 - 탐지: Macie (S3 PII 분류)
- **Attack-path** : 위 5개를 엮은 공격 경로
 - 탐지: 커스텀 상관 엔진

☑ 아키텍처 핵심 포인트

- **빨간 점선** = 크로스클라우드 공격 경로
 - (AWS 워크로드 침해 → 평문 Azure 자격증명 → Entra 신원 장악)

☑ 두 개의 APP

- **취약 App**(의도적 결함 · 스캔 대상)
- **관제 App**(관제 화면)
- **Agentless** — 취약 app과 관제 app은 직접 통신하지 않음
 - (스캐너의 read-only 스캔 결과만 관제가 조회)

Cross-Cloud 공격 시나리오 & 취약 App 설계

02. 탐지 · AI 분석

AWS 워크로드 침해가 **Azure 신원 장악**으로 번지는 크로스클라우드 공격 체인

01. product [AWS] 워크로드 침투

product 취약 이미지(KEV)
→ 컨테이너 장악, privileged
컨테이너로 측면이동



02. order [AWS] 자격증명 발견

order 파드 환경변수에 하드
코딩된 **Azure SP 자격증명**
탈취



03. member [AWS] 데이터 탈취

member 공개 S3에서 **PII 탈취**(합성 데이터, Macie가 사
전 분류)



04. Entra [Azure] 신원 장악

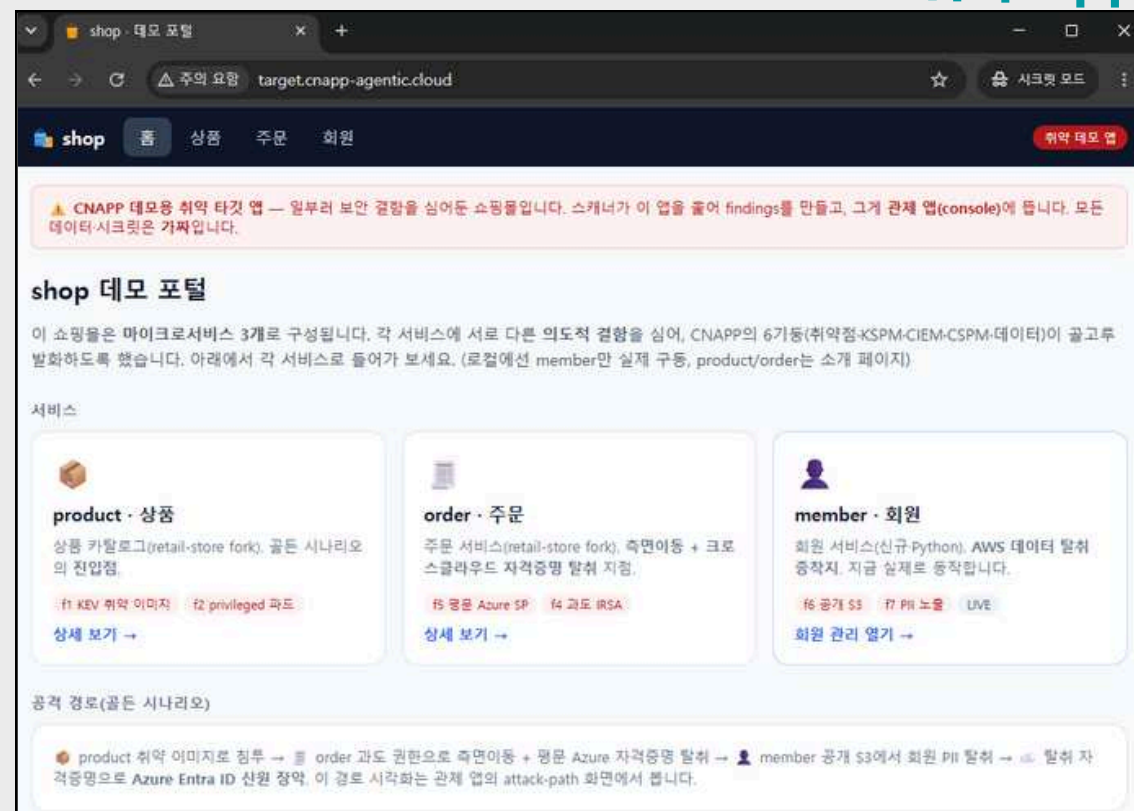
탈취한 자격증명으로 Entra
과도권한 앱 장악
→ **디렉터리 전체 통제**

크로스클라우드 경계(credential_theft)

→ 단일 클라우드 도구는 이 경계 횡단을 볼 수 없음

취약 App 설계 의도

취약 App



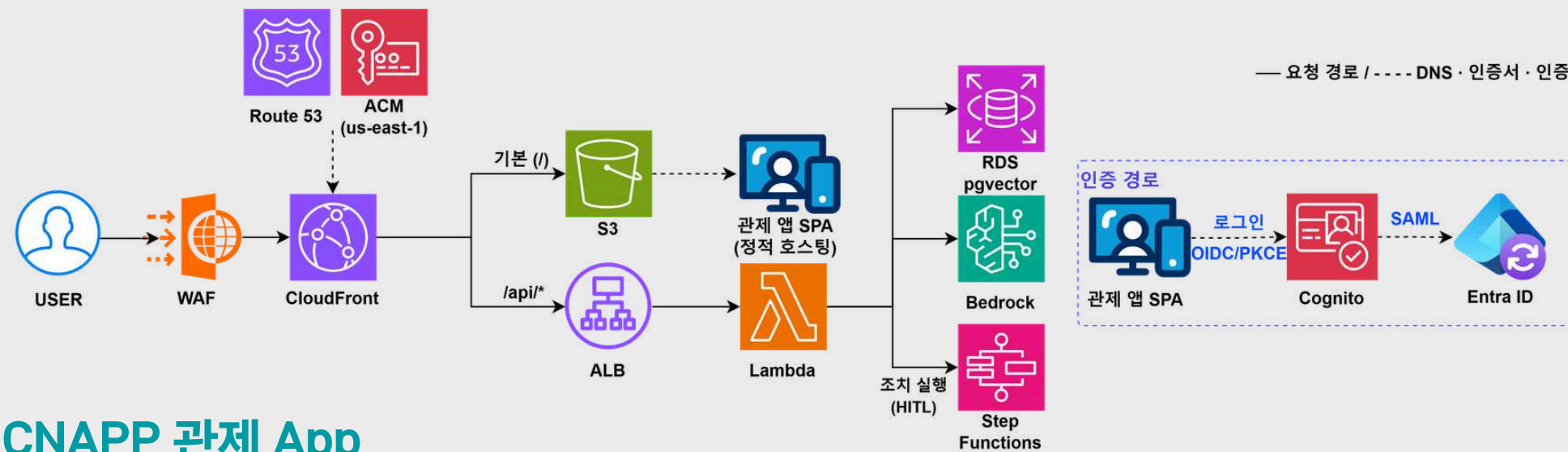
취약 App — 의도적으로 설계한 취약 환경

- **구성:** AWS 공식 retail-store-sample-app 기반(catalog·orders) + member 서비스는 직접 구현
- **결함:** TerraGoat·CloudGoat·EKS Goat 취약 패턴을 참고해 **laC에 직접 8종 (f1~f8)** 주입
- **원칙:** 앱 코드 버그가 아니라 인프라/설정 결함 — **기능은 최소, 결함 다양성은 최대**
- **Why:** CNAPP 플랫폼을 증명하려면 스캔할 "실제 취약점"이 있어야 함 → 의도적으로 결함을 심은 앱을 직접 배포 (= **findings의 출처**)

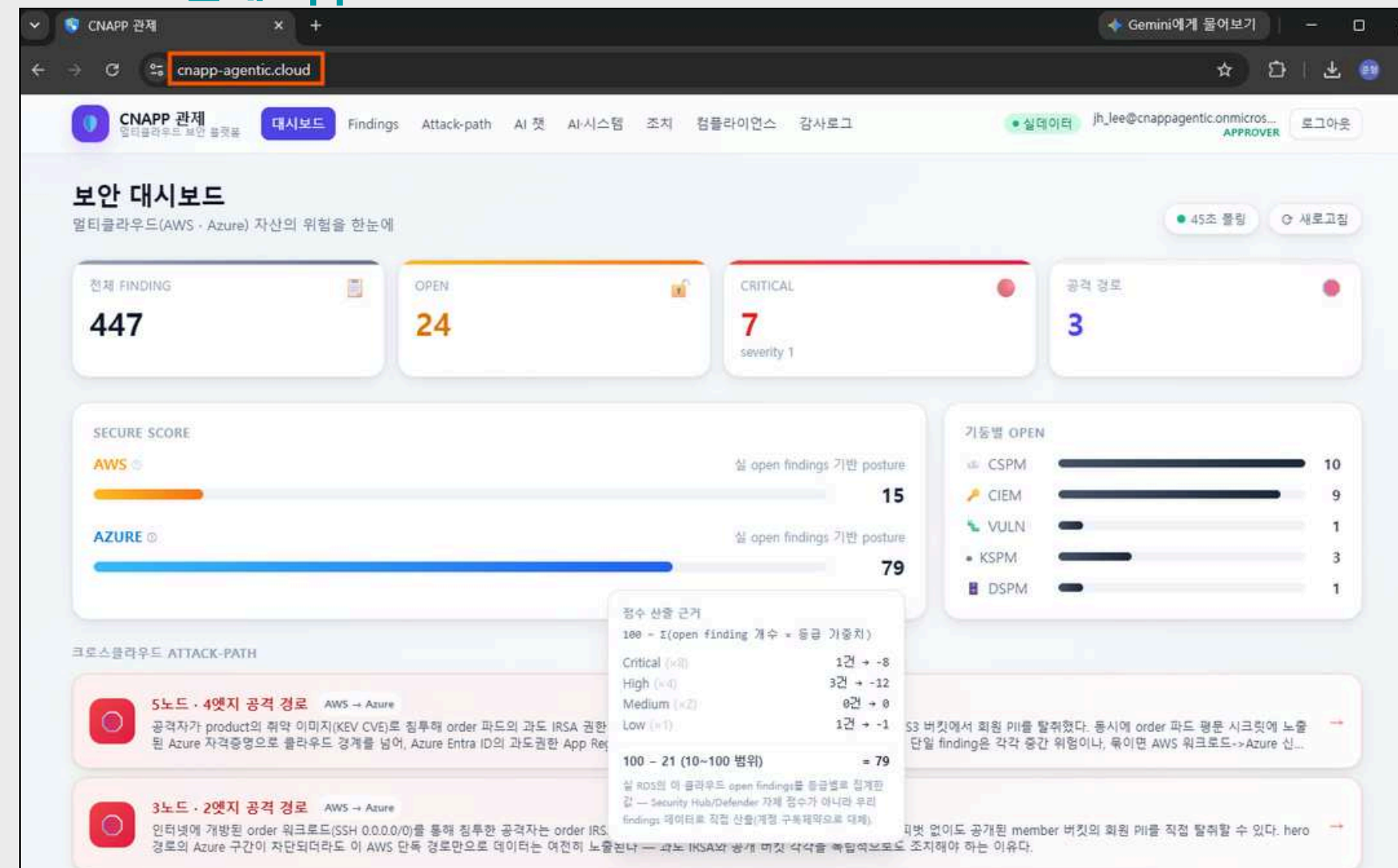
관제 콘솔 소개

02. 탐지 · AI 분석

취약 app을 감시하는 관제 콘솔 — 스캔·정규화·AI 조사·설명, 프로젝트의 모든 기능이 하나로 모이는 화면



CNAPP 관제 App



탐지·현황

- 대시보드: **Secure Score**·심각도별 KPI·크로스클라우드 **attack-path 배너**를 한 화면에 요약 → 상세 수치 **19p(성과)**
- Findings 목록: 스캐너가 올린 finding을 필터·정렬하고 AI 조사 상태를 컬럼으로 표시 → **12p(실 탐지 현황)**



AI 자율 조사·설명 (이 플랫폼의 차별점)

- Evidence(능동조사) 탭: AI가 **스스로 호출한 read-only API**와 판정 근거를 타임라인으로 표시 → **10p(Bedrock 실증)**
- AI 어시스턴트(/chat): **pgvector** 근거 검색 기반 보안 Q&A 챗봇 → **11p(RAG)**



공격 경로·시각화·관측

- Attack-path 그래프: 여러 공격 경로를 위험도순으로, AWS/Azure 레인으로 시각화(크로스클라우드 엣지 **빨강**) → **6p(공격 경로)**
- System 뷰: **Bedrock 24h** 사용량·데이터 파이프라인 건강 등 운영 지표 → **17p(운영 관측)**



그 외 실무 화면

- 컴플라이언스: ISMS-P 프레임워크 ↔ control 매핑, 통제별 충족률·PDF 리포트
- 조치(Remediation): **approver가 승인**하면 Step Functions로 **실제 조치 실행(HITL)** → **13p(거버넌스)**
- 감사(Audit): 스캔·판정·조치 이벤트를 시간순 **불변 로그로(S3 Object Lock)** → **13p(거버넌스)**

수집·정규화 파이프라인

02. 탐지 · AI 분석

보안 신호 파편화 - 스캐너마다 출력 포맷·심각도 기준이 제각각이면 담당자가 여러 콘솔을 오가야 함

이질적인 스캐너 출력을 하나의 **공통 스키마(OCSF-lite)**로 **정규화**해, 스캐너가 몇 개든 단일 콘솔·단일 우선순위로 **통합**



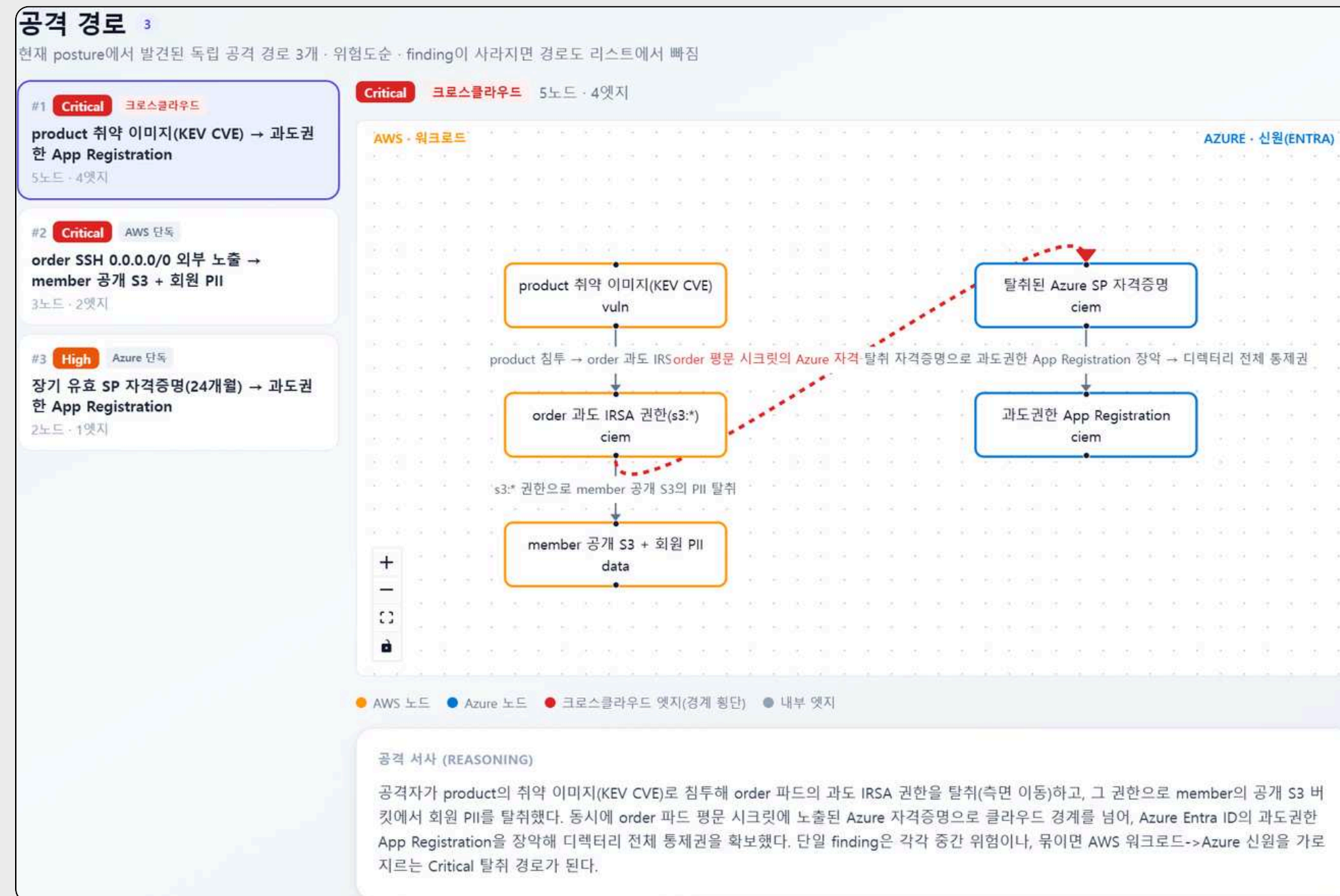
attack-path 상관 엔진

02. 탐지 · AI 분석

연관성이 안 보이던 개별 finding들이 묶이면 위험도가 격상

→ 흩어진 finding들을 상관 규칙을 통해 공격 경로(attack-path)로 체이닝하는 상관 엔진 구현

관계 콘솔의 attack-path 탭



공격 경로 리스트, AWS/Azure 그래프, AI가 생성한 Reasoning까지 한 화면에 표시



동시 공격 경로

- 3개의 attack-path가 동시에 탐지
 - 핵심 경로 (AWS-Azure 크로스클라우드)
 - AWS (데이터 탈취)
 - Azure (신원 장악)
- 위험도 순으로 정렬하여 표시하고, 하나의 finding은 여러 경로에 동시 소속 가능
 - ex) order의 과도한 IRSA 권한은 핵심 경로와 AWS 경로 둘 다 소속



상관 규칙

- 5개 상관 규칙이 개별 finding을 하나의 공격 체인으로 연결
 - KEV 취약 이미지 + 외부 노출 SG
 - 과도 IAM/IRSA 권한
 - 평문 Azure 자격증명 + 장기 유효 SP
 - 공개 S3 버킷의 PII 노출
 - Entra 위험 consent·과도 권한 App
- 다섯 개의 규칙이 전부 발화하면 완전한 attack-path가 완성

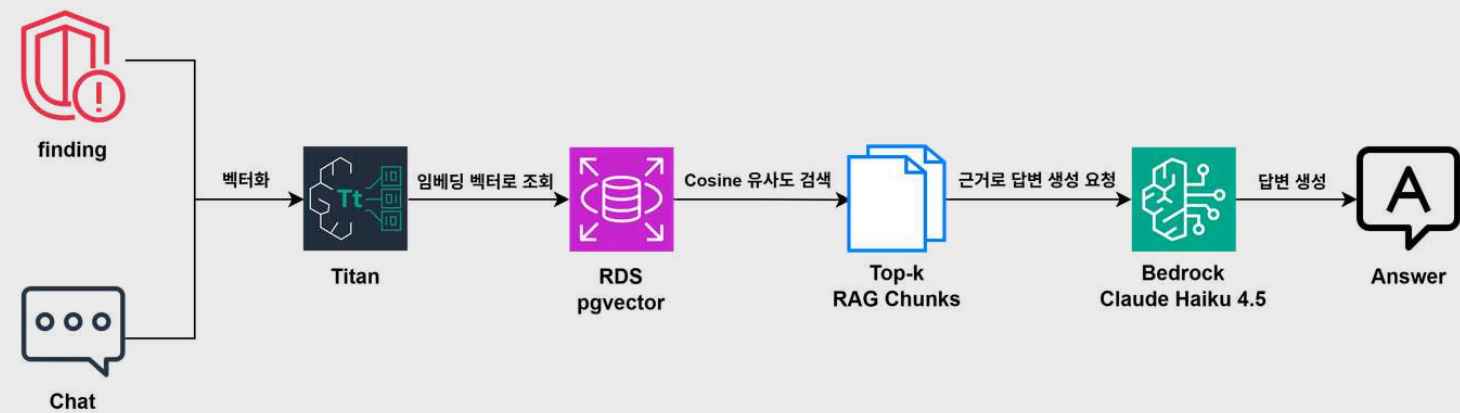


조치

- 조치 승인이 즉시 attack-path 소멸로 연결
- 공개된 S3 버킷에 대한 조치 완료 즉시 remediation Lambda가 상관 엔진을 비동기 재호출
- 버킷에 의존하던 경로가 재계산에서 빠져 RDS에서 삭제
- 콘솔 화면의 경로 3개 → 1개로 실시간 반영

finding이 왜 위험한지, 어떻게 고치는지를 LLM이 지어내지 않고, **지식 코퍼스**에서 벡터 검색한 **근거**로만 **대답 및 설명 (Hallucination 억제)**

RAG 파이프라인 흐름



- ① **질문 입력:** finding(자동 조사 대상) 또는 사용자가 /chat에 질문 입력
- ② **벡터화:** Titan Embed v2가 이 텍스트를 1024차원 벡터로 변환
- ③ **유사도 검색:** RDS pgvector가 이 벡터와 코사인 유사도가 가장 높은 지식 코퍼스 청크들을 검색
- ④ **Top-k 근거 선별:** 검색된 결과 중 유사도가 가장 높은 상위 k개(예: 4~5개)만 선별
- ⑤ **답변 생성:** Bedrock Claude Haiku 4.5가 이 근거 청크들만 참고해서 답변을 생성

Finding 상세 화면의 설명

AI 설명

능동조사 (Evidence)

★ 요약

member-pii-prod S3 bucket allows public access — control INTERNAL-S3-PUBLIC-001 위반(cspm) - 위험도 high

⚠ 왜 위험한가

member-pii-prod S3 버킷에 대한 5회 조사 결과, 공개 액세스 차단 설정이 비활성화되어 있고 버킷 정책에서 Principal:"*"를 허용하는 statement가 확인되어 공개 접근 위험이 confirmed(신뢰도 60%)되었다. 다만 ACL 수준에서는 공개 grantee가 없어 실제 노출 경로가 제한적일 수 있으며, 민감한 PII(이름, RRN 패턴)가 저장된 상태에서 정책 기반 공개 설정이 존재하므로 즉시 정책 검토 및 차단 설정 활성화가 필수다. 추가로 order IRSA 역할에 s3:*/Resource:* 와일드카드 권한이 인라인 정책으로 부여되어 있어 최소권한 원칙 위반이 확인되었으며, 이는 order 서비스의 권한 범위를 즉시 제한할 필요성을 나타낸다.

🔧 조치 방법

1. S3 퍼블릭 액세스 차단(Block Public Access) 즉시 활성화 2. Macie 분류 결과 기반 PII 파일 접근 제한 + 감사 로그 보존 3. IRSA 역할 권한 최소화(s3:* → 특정 버킷 ARN 한정)

- 해당 finding의 **요약, 위험한 이유, 조치 방법** 3단 구성
- AI 엔진의 조사 결과와 **RAG 근거**를 바탕으로 자동 생성
- 조사가 끝나면 사용자 요청 없이 즉시 콘솔에 반영

AI 어시스턴트 대화 화면

AI 보안 어시스턴트

RAG - 상 지식베이스

Titan 임베딩

pgvector 유사도 검색

Bedrock 답변

'member-pii-prod S3 bucket allows public access' 해당 finding에 대해 간단히 설명해줘

member-pii-prod S3 버킷 공개 접근 허용 Finding

문제 요약

member-pii-prod S3 버킷이 인터넷의 모든 사용자(Public)에게 접근 가능한 상태라는 보안 위협입니다.

위험성

- **높음:** PII(개인식별정보) 데이터가 포함된 버킷
- 비인가 사용자가 민감한 개인정보에 접근 가능
- 규정(GDPR, CCPA 등) 위반 가능성
- 데이터 유출 및 개인정보 보호법 위반

일반적인 원인

- S3 버킷 정책에서 Principal이 "*" (모두)로 설정
- ACL이 공개로 설정된 경우
- Block Public Access 설정이 비활성화 상태

해결 방법

1. 버킷 정책 검토 및 수정 - 특정 IAM 사용자/역할만 허용
2. ACL 변경 - Private으로 설정
3. Block Public Access 활성화 - 계정 수준에서 적용
4. 접근 제한 - 필요한 사용자에게만 권한 부여

이 Finding은 즉시 해결해야 할 심각한 보안 이슈입니다.

- finding 맥락이 **AI 어시스턴트** 대화창에 자동 연결되어 자유롭게 추가 질문 가능
- **Titan 임베딩 → pgvector 검색 → Bedrock** 순으로 근거를 찾아 답변 생성

✓ 근거 인용의 신뢰성

- 답변 문장은 **AI(Bedrock Claude Haiku)**가 작성
- 답변의 근거는 AI가 생성한 것이 아닌, 검색 단계에서 실제로 찾아낸 문서 목록
- 근거 없이 지어내는 **Hallucination** 방지

✓ 코퍼스 규모

- 보안 통제 15종에 대한 한국어 설명을 **26개 청크**로 나눠 사내 지식베이스로 구축
- 각 청크는 어떤 통제(control_id)에 대한 설명인지 태깅
- finding이 어떤 통제 위반인지만 알면 해당 설명으로 바로 매핑 가능

✓ RAG 검색 결과 사용자

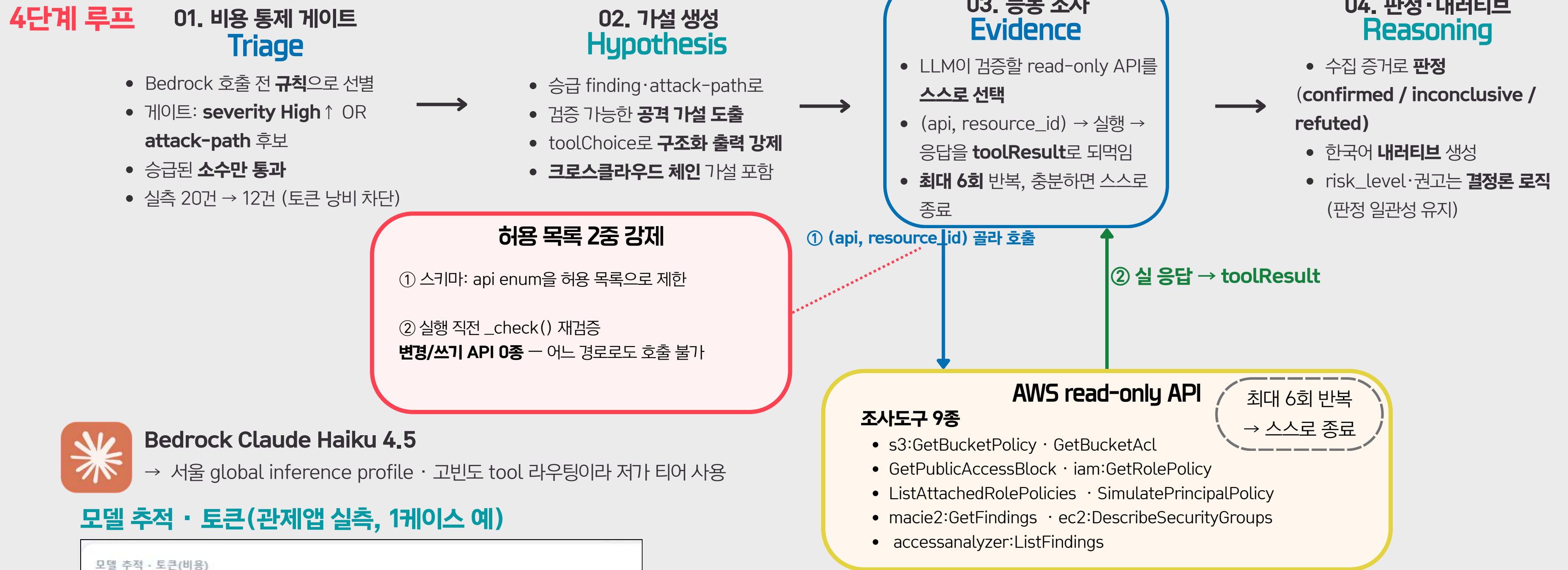
- **Finding 상세** 화면의 '설명' 탭에서 왜 위험한지·어떻게 고치는지를 자동으로 근거와 함께 보여줌
- /chat **AI 어시스턴트**에서는 사용자가 자유롭게 질문하면 같은 방식으로 근거를 찾아 대답

Agentic Engine 4단계 능동 조사 루프

02. 탐지 · AI 분석

기존 'AI 보안' = 데이터 요약 챗봇에 머뭇 → 이 엔진은 **Bedrock**이 read-only API를 스스로 호출·조사해 근거 기반 위험 판정

Orchestrator가 아래 4단계를 순차 호출하고 조사 케이스로 조립해 RDS에 저장



Bedrock Claude Haiku 4.5

→ 서울 global inference profile · 고빈도 tool 라우팅이라 저가 티어 사용

모델 추적 · 토큰(관제앱 실측, 1케이스 예)

모델 추적 · 토큰(비용)

triage: **haiku** · 0tok

hypothesis: **haiku** · 8730tok

evidence: **haiku** · 4802tok

reasoning: **haiku** · 994tok

챗봇 탈출의 단일 기준 -> 규칙이 정한 API가 아니라 **LLM이 스스로 선택·호출**

설계 설명이 아니라 실제 **AWS**·실제 **Bedrock**으로 관통 검증한 결과 — 라이브 콘솔 Evidence 탭

실측 결과

- ✓ 5회 read-only API를 스스로 호출
LLM이 스스로 호출한 read-only API (자율 tool use)
- ✓ 약 100원
1회 조사 비용 (Haiku 토큰, 화면 model_trace에 실측)
- ✓ E2E
배포→조사→판정→정리 전 구간 관통

RAG 챗봇 VS 능동조사엔진

- RAG 챗봇: “공개일 수 있습니다.” ← 지식 기반 추측
- 능동조사 엔진 : GetBucketPolicy 실행출
 - Principal:"*" 확인 → "공개 맞음" ← 증거 기반 판정

Bedrock 능동조사

- 공개 S3 finding 수신 → LLM이 스스로 s3:GetBucketPolicy·s3:GetPublicAccessBlock 호출 결정 (canned 아님)
- 실 API 응답 → 버킷 정책 **Principal:"*** 공개 statement + BlockPublicAcls=false 등 PAB 미설정 확인
- 증거 기반 **confirmed** 판정(확증률 60%) → 케이스에 tool 호출 타임라인·판정·model_trace(토큰)·지식베이스 근거(RAG) 기록

①Triage

① 트리아지 게이트

ESCALATE escalate — attack_path_id!=null(경로 소속) · severity_id=1(High ↑)

②Hypothesis

② 가설 (HYPOTHESIS)

aws:s3_bucket:member-pii-prod-066107819776 버킷이 공개 접근을 허용하면, 외부 공격자가 민감한 PII 데이터(이름, RN 패터)를 직접 탈취할 수 있다.

aws:iam_role:cnapp-agentic-github-ci 역할이 iam:PassRole 와일드카드 권한을 가지면, 공격자가 이 역할을 통해 더 높은 권한의 역할을 생성하거나 기존 역할에 권한을 부여하여 권한 상승을 수행할 수 있다.

aws:eks_podshop/product 파드가 특권 컨테이너로 실행되고 CVE-2024-3094 취약점을 포함하면, 공격자가 이 취약점을 악용하여 EKS 클러스터 내에서 권한을 상승시키고 다른 워크로드에 접근할 수 있다.

aws:iam_role:cnapp-agentic-target-order-irsa 역할이 Resource:* 에 대한 s3:* 와일드카드 권한을 가지면, 공격자가 이 역할을 탈취하여 모든 S3 버킷의 데이터를 열거, 읽기, 삭제할 수 있다.

azure:service_principalb2c3d4e5-7777-8888-9999-aaaabbbbbb 서비스 주체의 자격증명이 24개월 유효기간으로 설정되고, aws:secret_plaintextshop/order/AZURE_SP_CRED에 평문으로 저장되면, 공격자가 AWS에서 Azure 자격증명을 탈취하여 Azure 리소스에 대한 크로스클라우드 접근 권한을 획득할 수 있다.

③Evidence

③ 능동 수집 증거 · READ-ONLY API 호출 5회 호출 · 확증률 60%

1

s3:GetBucketPublicAccessBlock aws:s3_bucket:member-pii-prod-066107819776

BlockPublicAcls=False, RestrictPublicBuckets=False — public access block 미설정

2

s3:GetBucketAcl aws:s3_bucket:member-pii-prod-066107819776

공개 grantee 없음(ACL 정상)

3

s3:GetBucketPolicy aws:s3_bucket:member-pii-prod-066107819776

Principal:"*" 허용 statement 확인 — 공개 버킷

4

iam:GetRolePolicy aws:iam_role:cnapp-agentic-target-order-irsa

인라인 'order-s3': s3:* / Action 와일드카드 + Resource:* — 과도 권한

5

iam:ListAttachedRolePolicies aws:iam_role:cnapp-agentic-target-order-irsa

부착 관리형 정책 0개(광역 정책 없음): -

④Reasoning

④ 판정 (REASONING)

— 확증률 60% —

5회 조사를 통해 member-pii-prod S3 버킷의 공개 접근 허용과 order IRSA 역할의 과도한 권한을 확인했다. 버킷 공개 액세스 차단이 비활성화되어 있고 버킷 정책에서 Principal:"*"를 허용하는 statement가 발견되어 공개 노출 위험이 실제로 존재한다. 동시에 order 역할에 s3:* / Action 와일드카드와 Resource:* 조합이 인라인 정책으로 설정되어 있어 민감한 PII 데이터에 대한 과도한 접근 권한이 부여된 상태다. 현재 신뢰도 60%는 ACL 상 공개 grantee가 없는 점으로 인한 것으로, 버킷 정책 기반의 공개 노출과 권한 과다 부여는 긴급 대응이 필요한 상태다.

지식베이스 근거 INTERNAL-S3-PUBLIC-PII INTERNAL-DATA-PII-EXPOSED-IRSA INTERNAL-IAM-OVERPRIV-IRSA

규칙이 정한 API가 아니라, LLM이 finding을 읽고 조사 도구를 스스로 선택·호출

능동조사(Evidence) 탭 화면

AI 설명 능동조사 (Evidence)

AI 능동조사 판정 confirmed

60% 확증률

AI 에이전트가 5회 read-only API를 스스로 호출해 증거를 수집하고 판정했습니다.

확증률

확증 도구수 ÷ 전체 (결정론적 산식)

관제 콘솔 실 탐지 결과

02. 탐지 · AI 분석

실제 **스캐너**를 라이브로 돌려 실 계정·실 클러스터를 read-only로 스캔한 결과, 실제 **finding**이 콘솔에 반영
→ **CNAPP 보안 플랫폼의 실동작 검증** (CSPM·CIEM·Vuln·KSPM·DSPM)

CSPM

Prowler · Security Hub · Config

Critical AWS CSPM

open

member-pii-prod S3 bucket allows public access

aws:s3_bucket:member-pii-prod-066107819776

control: INTERNAL-S3-PUBLIC-001

sources: securityhub:S3.8, config:s3-bucket-public-read-prohibited, prowler:s3_bucket_public_access

attack-path: a0000000-0000-4000-8000-000000000001

- AWS 계정 전체 리소스를 대상으로 CSPM 체크 수행 (Prowler는 GitHub Actions로 매일 자동 스캔)
- 같은 공개 S3 버킷을 세 도구가 각각 탐지 → dedup_key 기준 finding 1건으로 병합

KSPM

kube-bench

High AWS KSPM

open

Ensure that the cluster-admin role is only used where required (Automated) (shop/product)

aws:eks_pod:shop/product

control: INTERNAL-KSPM-CLUSTERADMIN-001

sources: kube-bench:4.1.9, kube-bench:4.1.5, kube-bench:4.1.7, kube-bench:4.1.1, kube-bench:4.1.11 +7개 더보기

- EKS 클러스터 설정을 in-cluster job으로 CIS Amazon EKS Benchmark 실행
- RBAC 관련 체크 12개(4.1.1~4.1.12)가 하나의 finding에 누적

CIEM (AWS)

IAM Access Analyzer

Critical AWS CIEM

open

IAM Access Analyzer: AWS::S3::Bucket externally reachable (공개(Public))

aws:s3_bucket:member-pii-prod-066107819776

control: INTERNAL-IAM-OVERPRIV-001

sources: access-analyzer:AWS::S3::Bucket

- AWS 계정의 IAM·S3 버킷 정책에서 외부 신원이 접근 가능한 리소스를 자동 분석
- GitHub Actions가 정기 실행 → EventBridge 경유로 파이프라인 합류

CIEM (Azure)

Prowler

High AZURE CIEM

open

Entra authorization policy disallows user consent for applications

azure:other:authorizationPolicy

control: INTERNAL-ENTRA-RISKY-CONSENT-001

sources: prowler:entra_policy_user_consent_for_verified_apps, prowler:entra_policy_restricts_user_consent_for_apps

- Azure Entra 테넌트의 앱 등록·사용자 동의 정책을 Prowler로 스캔
- AWS와 동일한 Prowler를 Azure에 실행, prowler-sp의 Federated Credential로 키 없이 인증

Vuln

Trivy · Inspector

Critical AWS VULN

open

KEV-listed CVE (CVE-2024-3094) in product image

aws:eks_pod:shop/product

control: INTERNAL-VULN-KEV-001

sources: inspector:CVE-2024-3094, trivy:CVE-2024-3094

attack-path: a0000000-0000-4000-8000-000000000001

- ECR에 push된 product 컨테이너 이미지를 in-cluster job으로 스캔
- 동일 CVE-2024-3094를 Trivy·Inspector가 독립 탐지 → 하나의 finding에 소스 누적

DSPM

Macie

High AWS DSPM

open

SensitiveData:S3Object/CustomIdentifier detected in member-pii-prod-066107819776 (200건)

aws:s3_bucket:member-pii-prod-066107819776

control: INTERNAL-DATA-PII-EXPOSED-001

sources: macie:SensitiveData:S3Object/CustomIdentifier, macie:SensitiveData/PII

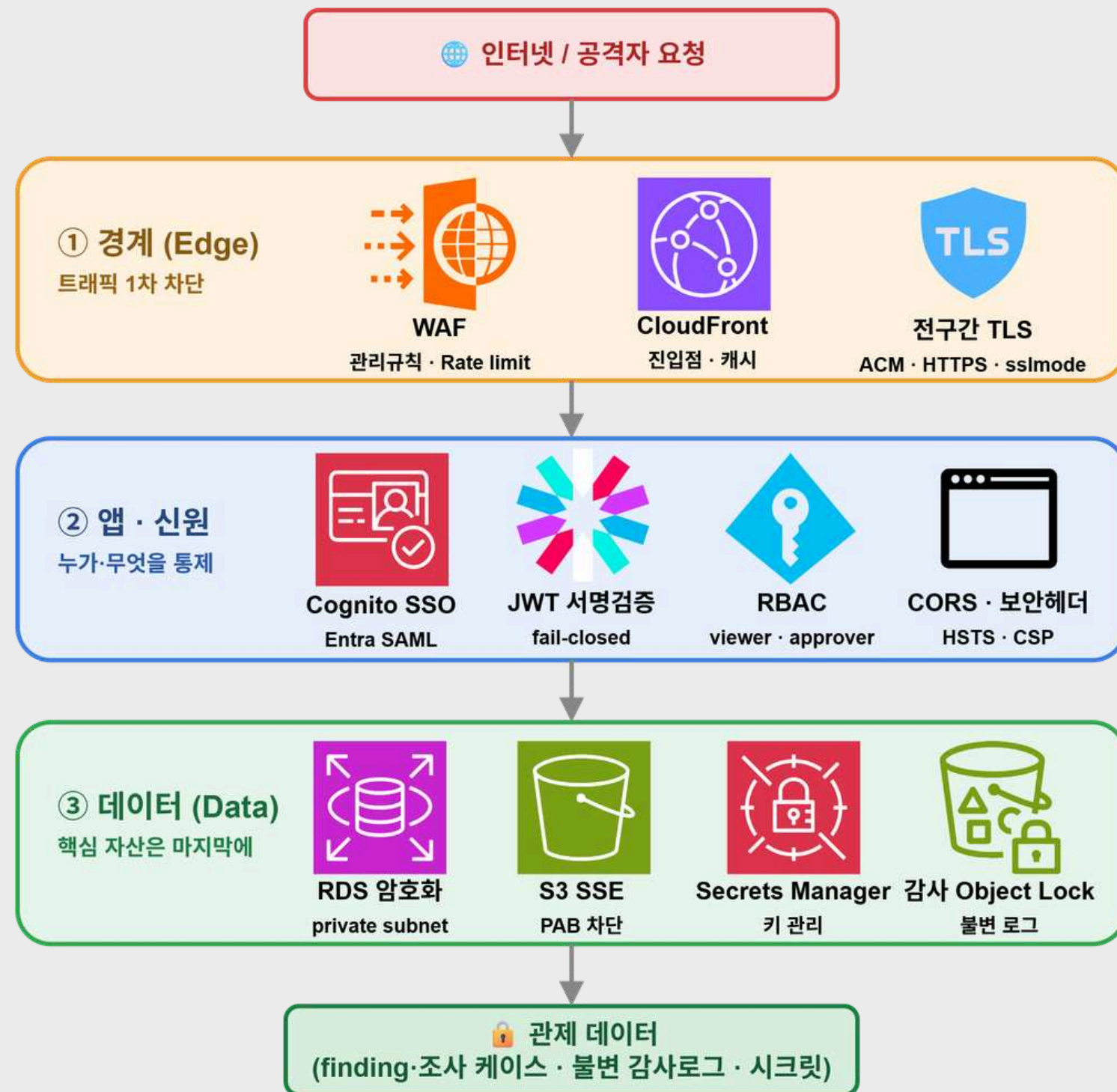
attack-path: a0000000-0000-4000-8000-000000000001

- 관리형 식별자에 한국 RRN이 없어 커스텀 데이터 식별자를 직접 정의 (200건 탐지)
- SensitiveData는 Security Hub로 relay되지 않아 macie2 API 직접 호출 경로 신설

보안 거버넌스 & 플랫폼 자기방어

03. 인프라 · 운영

제품이 타깃에서 잡아내는 결함을 플랫폼 자신에게도 적용 — WAF·SSO·키리스·불변 감사 + Read-only/HITL, JWT 서명 미검증 취약점도 스스로 발견·봉합



플랫폼 자기방어

☑ 경계 · CloudFront WAF

- WAFv2 WebACL (AWS 관리규칙 CommonRuleSet·KnownBadInputs + IP당 5분 2000 Rate limit)
- 실검증: XSS 페이로드가 CrossSiteScripting_QUERYARGUMENTS로 실제 차단 (CloudWatch 지표 확인)

☑ 앱·신원 · 키리스 인증

- 장기 액세스 키 미사용
- CI는 GitHub OIDC→IAM Role, 파드는 IRSA로 임시 자격증명만
- Azure도 prowler-sp Federated Credential로 키리스 (비밀번호는 Secrets Manager로 관리)

☑ 앱·신원 · SSO 페더레이션

- Entra SAML IdP → Cognito(OIDC/PKCE) → 그룹 GUID 클레임으로 viewer/approver RBAC
- console-backend는 aws-jwt-verify로 JWKS 서명 · issuer · audience · exp 검증 (fail-closed)

☑ 데이터 · 불변 감사로그

- 모든 조치를 Object Lock GOVERNANCE 모드 버킷에 기록
 - → 삭제·변조 차단 (우회하려면 별도 특권 필요)

에이전트 거버넌스

☑ Read-only First

- 에이전트는 기본적으로 조회만
- Evidence의 allowlist가 스키마·실행 2중으로 강제해 변경/쓰기 API는 원천 차단

☑ HITL 승인 경로

- 실제 변경(조치)은 사람 승인 후에만
- approver가 콘솔에서 승인 → remediation_requests INSERT → Step Functions StartExecution → finding remediated → Secure Score 상승
- terraform drift 0(재apply 시 안 꼬이는 조치만 데모 — S3 SSE·ECR scan)

제품이 취약App에서 잡는 결함을 플랫폼 자신에게도 적용 — JWT 서명 미검증 권한상승 취약점을 스스로 발견·봉합

종량제 리소스를 상시 켜두지 않고, 언제든지 처음부터 재현 가능하도록 인프라 자체를 Terraform으로 코드화
안전하게 반복 가능한 apply → destroy 구조 설계

① 정방향 apply → shared→
karpenter→target→backend→c
onsole→monitoring 순서를 코드가
강제

```
PS F:\lee_jh\aws_클라우드\개인보안프로젝트\cnapp-agent\c> .\infra\deploy.ps1 -Action apply -AutoApprove
== terraform apply | order: shared -> karpenter -> target -> backend -> console -> monitoring ==
--- [shared] terraform apply ---
```

② 역방향 destroy —
monitoring→console→backend
→target→karpenter→shared 자
동 역순

```
== terraform destroy | order: monitoring -> console -> backend -> target -> karpenter -> shared ==
(deploy is reverse: monitoring first -> shared last; Grafana Ingress ALB is auto-released before monitoring destroy)
--- [monitoring] terraform destroy ---
--- [console] terraform destroy ---
--- [backend] terraform destroy ---
--- [target] terraform destroy ---
--- [karpenter] terraform destroy ---
--- [shared] terraform destroy ---
== done: terraform destroy (monitoring, console, backend, target, karpenter, shared) ==
```

③ monitoring destroy 전
Grafana Ingress(ALB) 자동 해제
→ ALB 고아 방지

```
[monitoring] Grafana Ingress released (ALB deleted) -- safe to destroy monitoring
[karpenter] orphan-node sweep: none found (controller deprovisioned cleanly)
```

④ destroy 후
EKS·RDS·VPC·Lambda·ALB 전부 0

```
$R="ap-northeast-2"
"== EKS =="; aws eks list-clusters --region $R --query clusters --output json
EKS : []
RDS : []
VPC : []
Lambda : []
ALB : []
```

6레이어 · 리소스 374개를 apply → 검증 → destroy · 잔존 리소스 0

✓ 순서를 코드로 강제한 레이어드 Terraform

- shared·karpenter·target·backend·console·monitoring
6레이어
- 하위 레이어가 상위를 terraform_remote_state로 참조해
의존 방향을 명시적으로 고정
- deploy.ps1이 apply는 정방향, destroy는 역방향으로 순회하며
실패 시 즉시 중단(상태 보호)
- Karpenter destroy 후 고아 스팟 노드까지 자동 종료

✓ 비용을 설계에 내재화한 네트워크

- 10.20.0.0/16, 2 AZ(ap-northeast-2a/2c), public/private
서브넷 분리
- NAT Gateway(\$32/월) 대신 NAT Instance(t4g.micro,
IMDSv2 강제)로 대체
- S3는 Gateway Endpoint로 NAT를 우회

✓ 탄력적인 Compute·Data 계층

- EKS 1.34(관리형 노드그룹 spot, scale 0~2)
- Karpenter(Spot 우선, On-demand 풀백)
- HPA로 트래픽에 따라 자동 파드 확장·회수
- RDS PostgreSQL 16(t3.micro)
- pgvector를 private 서브넷에 두고 Secrets Manager로만 접근

LLM 호출 전 규칙 게이트 · 케이스별 토큰 귀속 · 경량 대체와 apply→destroy 자동화로 비용을 아키텍처에 내장하고, 실사용·크레딧·순액 3축 관측으로 소비를 상시 추적

전략	왜 골랐나(트레이드오프)	효과
경량 대체(NAT Instance·RDS t3.micro+pgvector)	관리형 편의보다 비용 — 데모 규모엔 경량이 합리적	NAT 월 ~\$32 → ~\$3 (약 90% ↓)
Mock-first 개발	인프라 없이 2인 병렬 개발 - 계약 스키마를 먼저 고정하고 이 후 실 연동으로 전환	빌드 기간 클라우드 인프라 미가동
Apply → Test → Destroy	상시 가용성 포기, 월 과금 → 시간 과금	배포 스크립트(deploy.ps1)가 6레이어 순서까지 강제 — 정리 누락 방지
모델 티어링 + Triage 게이트	저가 모델 + 승급된 것만 조사(비용 통제를 제품에 내장)	실측 20건 → 12건만 승급 · 조사 1회 ~100원
비용 가시성 자동화	줄이는 것만으론 부족 — 쓰는 만큼 보여야 통제된다	일일 비용 알림 · Bedrock 시간당 임계 알람 · 케이스별 토큰 귀속 (EMF)

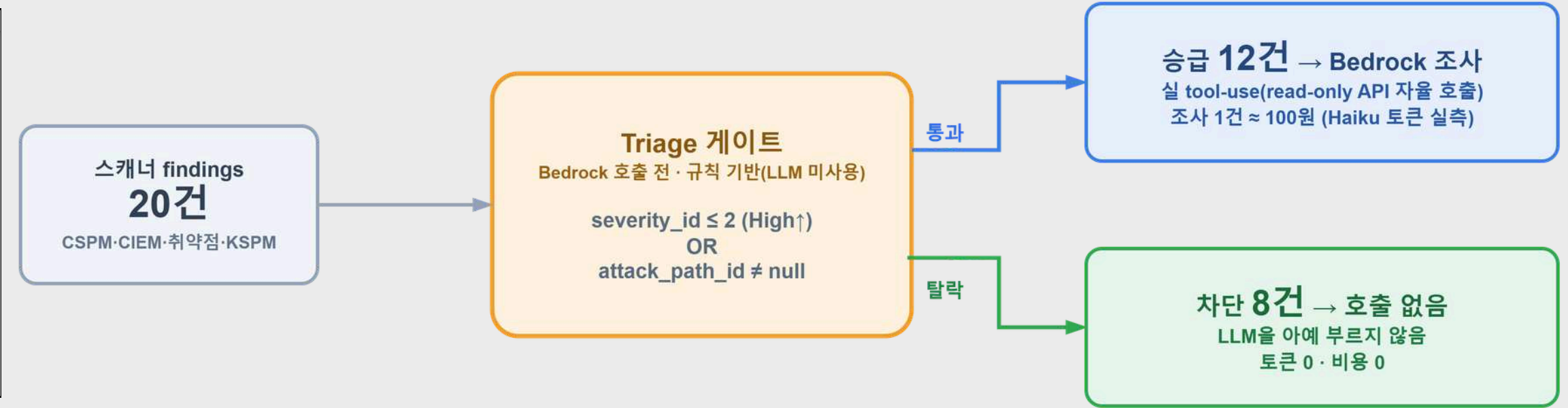
💡 일일 비용 알림을 순액 기준으로 두면 프로모션 크레딧이 실사용을 상쇄해 매일 \$0으로 보인다는 걸 확인 — 알림을 실사용·크레딧·순액 3줄로 재구성해 실제 소비를 정확히 추적

Triage 게이트 — 고비용 LLM 추론 앞에 저비용 결정론적 필터 배치

```
agentic > engine > evidence > triage.py > triage
def triage(finding: dict) -> TriageResult:
    """단일 finding 트리아지. escalate 여부 + 우선순위 점수."""
    sev = int(finding.get("severity_id", 5))
    on_path = finding.get("attack_path_id") is not None
    status = finding.get("status", "open")

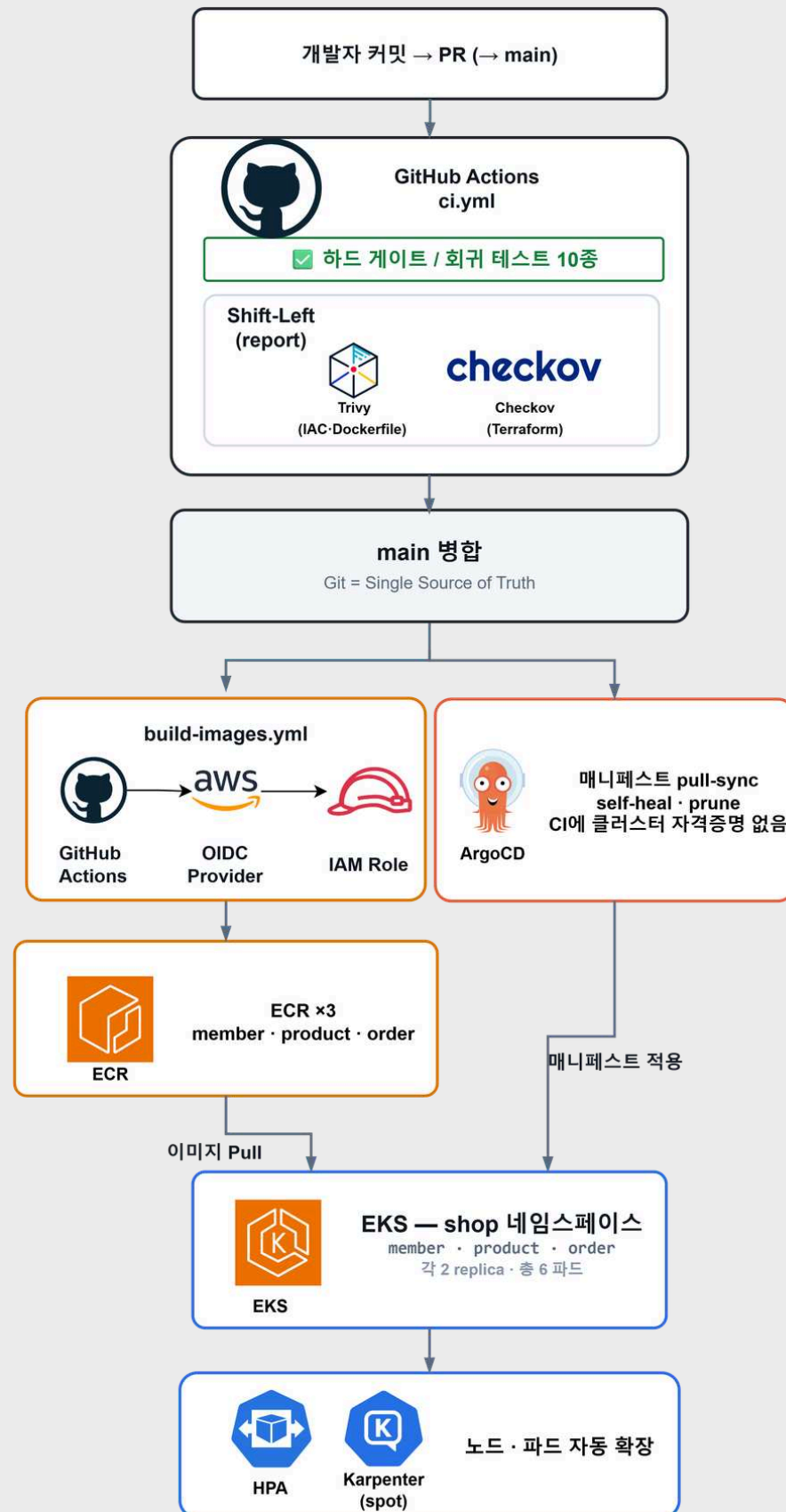
    # 게이트(4.4 2-pass): High↑(sev<=2) 또는 attack-path 후보만 승급
    escalate = status == "open" and (sev <= 2 or on_path)

    # 우선순위 점수 - 심각도(낮을수록 심각) 역수 + 경로 가산. 이미 계산돼 있으면 skip
    if finding.get("priority_score") is not None:
        score = float(finding["priority_score"])
    else:
        score = (6 - sev) / 5.0 # sev1+1.0 ... sev5+0.2
        if on_path:
            score = min(1.0, score + 0.25) # 경로 소속이면 가산
```

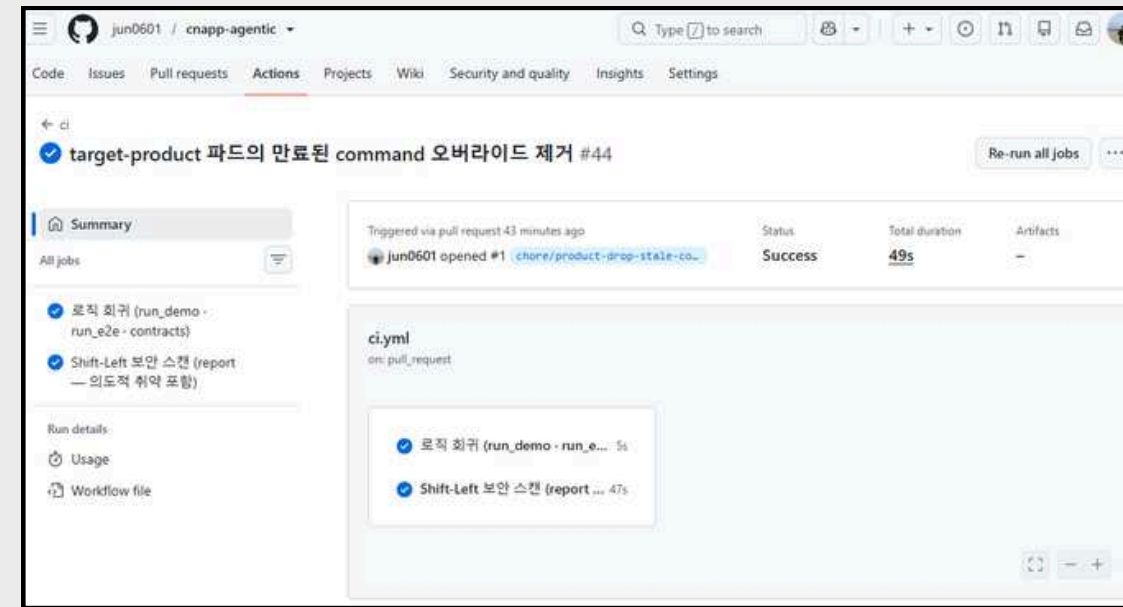


종량제·구독형 보안 서비스(Config·Security Hub·Inspector·Macie 등)는 상시 켜지 않고 검증 세션에서만 켜고 destroy — 상시 비용을 시간 비용으로 전환

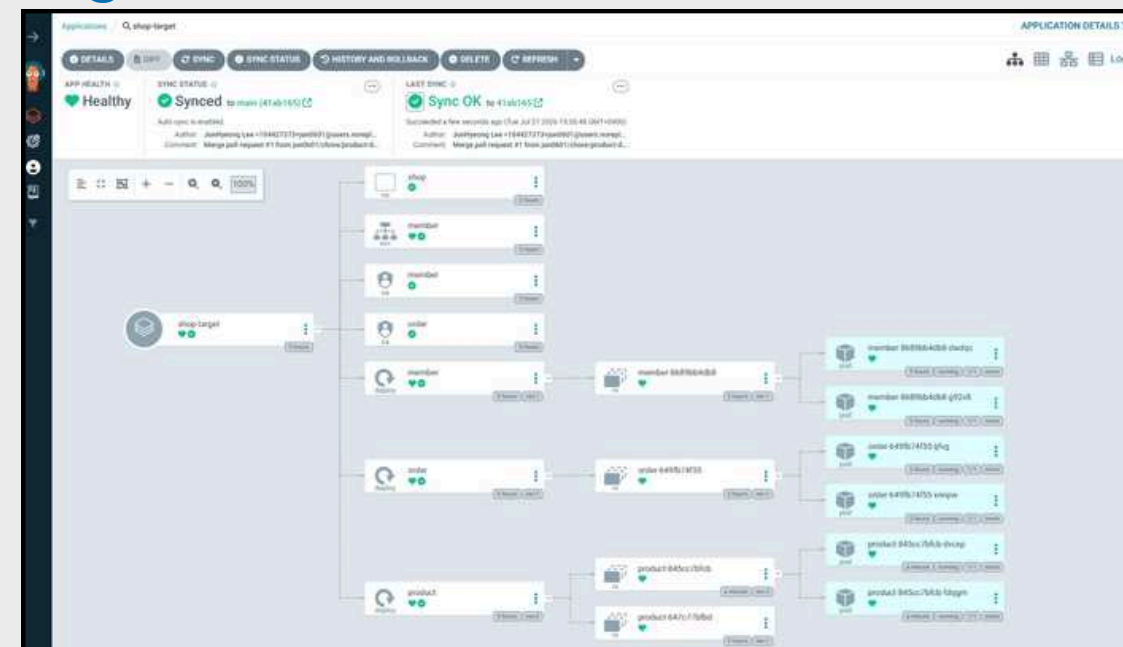
커밋은 회귀 게이트를 통과해야만 병합되고, 병합 이후 배포에는 사람이 개입하지 않는 구조



Github Action



ArgoCD



✓ CI: 병합 전 하드 게이트

- PR(→main)에서 contracts 계약 4-assert + 회귀 10종(영역별 run_demo 9개 + run_e2e)이 전부 **exit 0**이어야 병합 가능
- 함께 도는 Shift-Left 스캔은 Trivy(IaC·Dockerfile)·Checkov(Terraform)
- 의도적 취약 타깃을 품은 레포라 report 모드(발견해도 CI 막지 X)**
※ Trivy = 배포 전 미스컨피그(Shift-Left), 13p는 배포 후 CVE — 같은 도구·다른 시점

✓ CD: pull 기반 GitOps

- ArgoCD가 3분 주기로 Git 매니페스트를 당겨 클러스터 상태와 비교
- self-heal**은 kubectl로 직접 바꾼 값을 Git 상태로 되돌리고, **prune**은 Git에서 지운 리소스를 클러스터에서도 삭제
- CI엔 kubeconfig·클러스터 토큰 X**

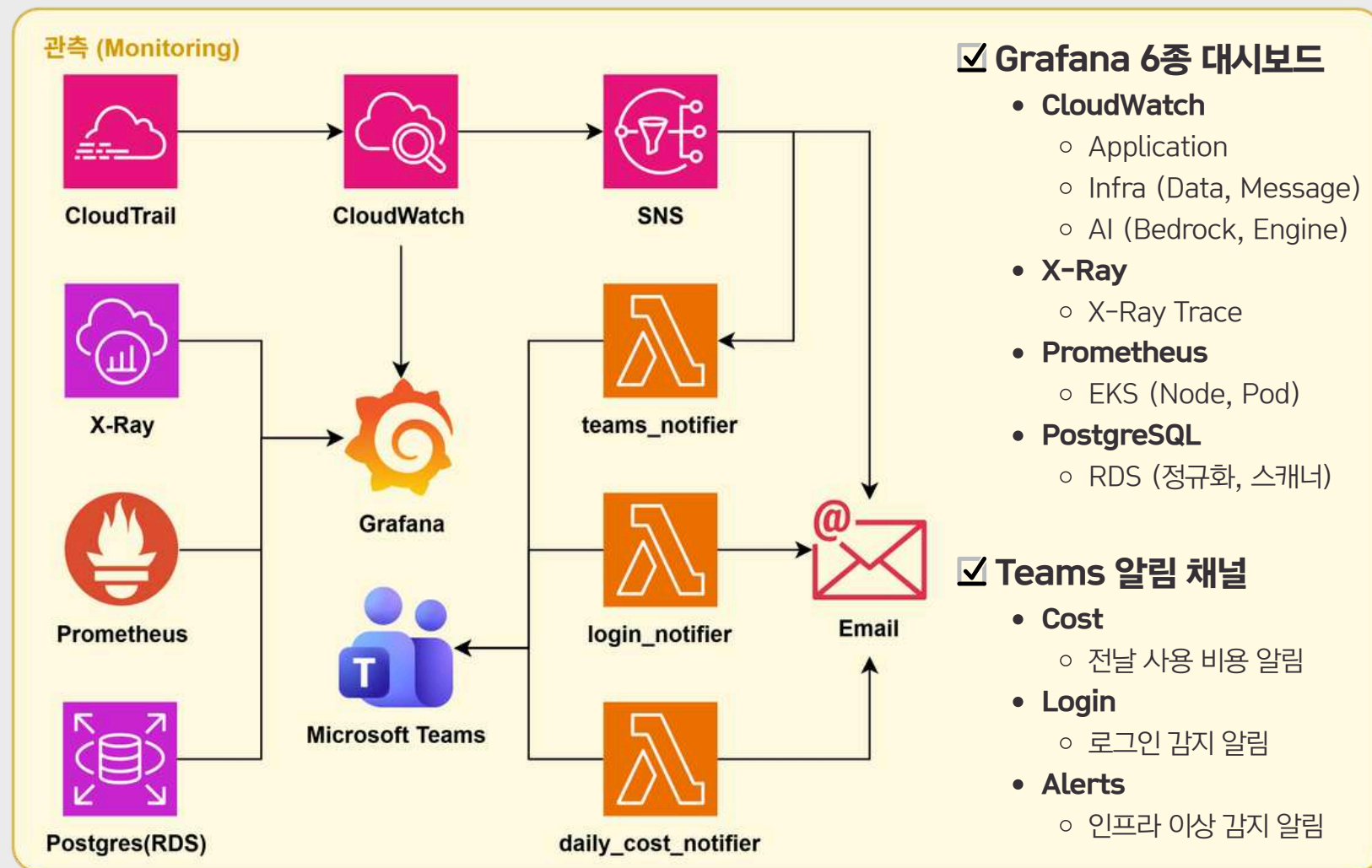
✓ 키리스 빌드 & 자동 운영

- GitHub OIDC** → **IAM Role**로 **장기 키 없이** ECR x3 (member·product·order) 빌드·푸시
- HPA**(CPU 70%, 파드 2→4) + **Karpenter**(스팟 자동 프로비저닝·유휴 회수)
- 정기 스캔**(prowler 02:00 · access-analyzer 02:30) → S3 → ingest Lambda

COMMIT → CI 게이트 → ECR → ArgoCD → EKS

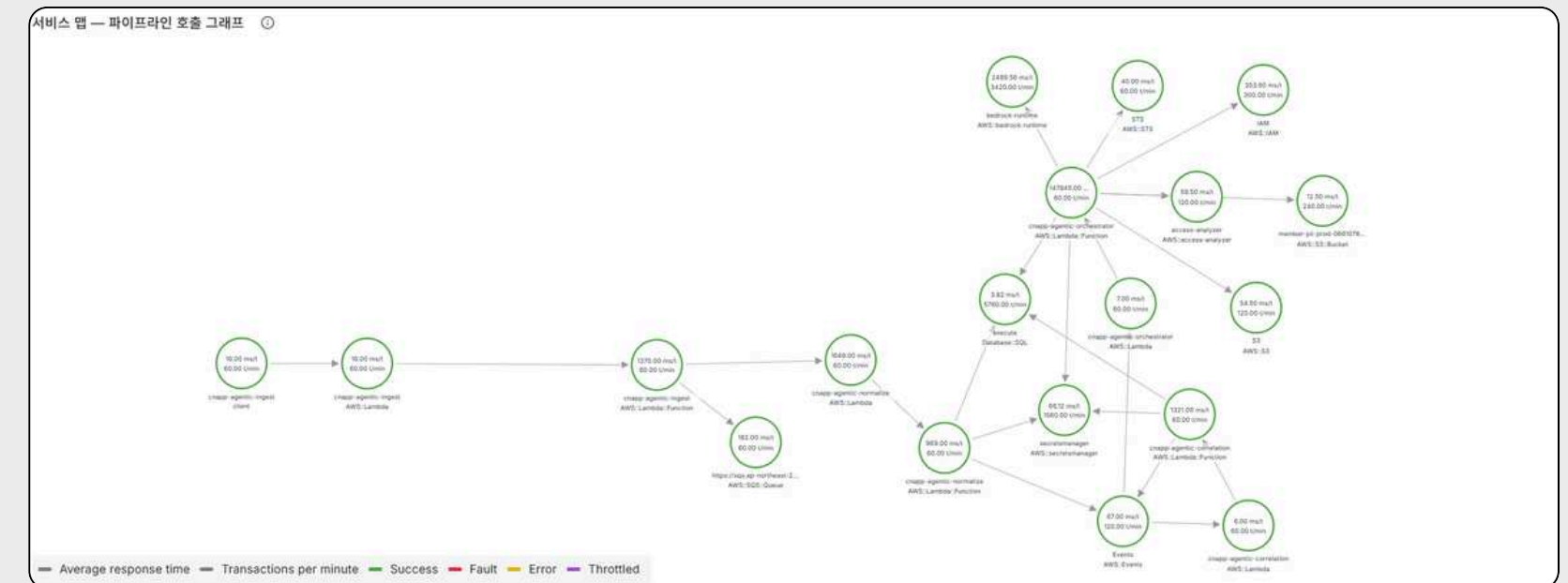
AI 에이전트가 실제로 호출한 API 하나까지 **X-Ray**로 추적 및 엔진 루프의 비용·행동 패턴 (토큰·tool 호출 등)을 **Grafana** 커스텀 지표로 계측
→ 단순 인프라 관측을 넘어, AI 에이전틱 루프 자체를 관측

전체 관측 흐름



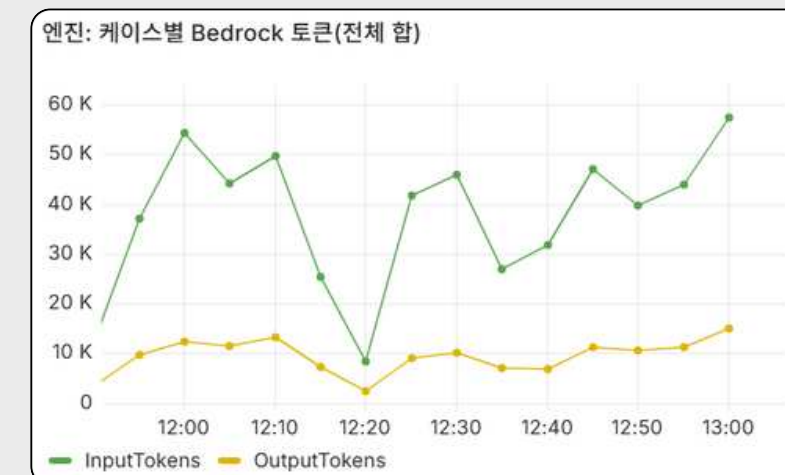
- **CloudTrail · CloudWatch**가 AWS 활동 및 지표 수집
- **SNS, Lambda** 함수를 통해 **Teams** · 이메일로 알림 전송
- **Grafana**가 **Prometheus · CloudWatch · PostgreSQL · X-Ray** 4개를 데이터소스로 연결해서 한 화면에서 관측

X-Ray 추적

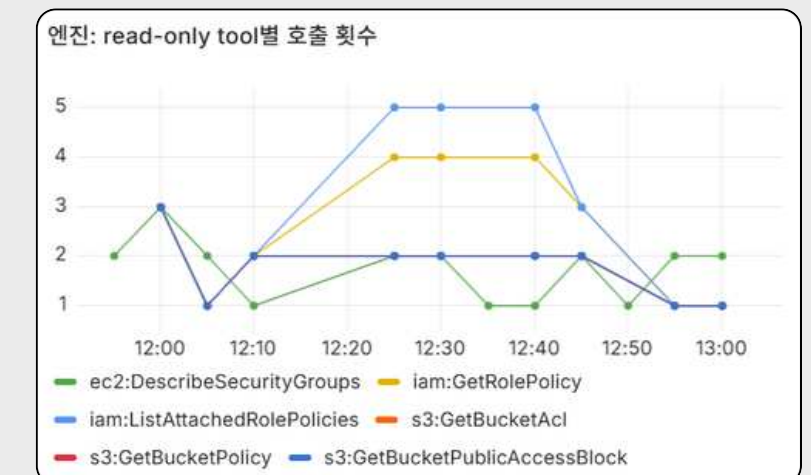


- **ingest→normalize→correlation→orchestrator** 전 구간이 트레이스로 연결
- AI가 자율적으로 호출한 API까지 포함해 파이프라인 전체 흐름을 한 화면에서 관측

능동 조사 엔진 계측



- 능동 조사 과정에서 쓰인 **Bedrock 토큰**을 합산해 추이로 표시
- AI 판단 1건당 비용을 시간 순으로 관측



- AI가 조사 과정에서 실제로 호출한 **read-only API** 별 호출 횟수를 시간순으로 표시
- 조사 과정의 행동 패턴 관측

라이브 환경에서 직접 부딪힌 문제 4건 — IAM · EKS · 멀티클라우드 · 데이터 파이프라인 각 계층

X-Ray Error 노드 규명

- 증상** - X-Ray 서비스 맵에 Error 노드 5개
애플리케이션은 중립 결과로 강등되어 정상 종료 → 콘솔엔 아무 문제도 안 보임
- 추적** - batch-get-traces로 실 예외 조회 → 실버그 2건 선별
 - ① 정책에는 권한이 있는데 s3:GetPublicAccessBlock 전부 AccessDenied
→ 액션명을 대조해보니 AWS가 요구하는 IAM 액션명은
s3:GetBucketPublicAccessBlock (API 오퍼레이션명 ≠ IAM 액션명)
 - ② iam:GetRolePolicy가 finding에 기록된 리소스명으로 호출되었으나,
실제 배포된 역할명과 달라 NoSuchEntity
- 조치** - 액션명 6곳 일괄 정정 + finding 리소스명 실물 교정
확증률 33% → 60%, 판정 inconclusive → confirmed

Prometheus 미기동

- 증상** - ArgoCD Application이 계속 OutOfSync, CRD 4종과 Prometheus
커스텀 리소스가 생성되지 않음
- 추적** - ServerSideApply 옵션만론 미해결 → CRD 생성 자체를 확인
→ 대형 CRD가 kubectl apply의 262144바이트 annotation 제한 초과로 생성 실패
→ CRD를 수동으로 설치해도 안 됨
→ operator 파드가 “CRD 없음”을 캐싱해 재시작 전까지 영원히 재확인하지 않음
- 조치** - ArgoCD PreSync 혹은 공식 CRD를 --server-side apply로 선행 설치해
크기 제한 · 캐싱 원천 차단 → 재구축해도 매번 수동 개입 없이 그대로 Synced
(⚠ CRD 버전이 차트 내부 operator 버전과 어긋나면 기능은 되나 diff가 남음)

Azure 구독 제약 오판

- 증상** - az role assignment create/list가 이 구독에서 MissingSubscription으로 실패
Global Admin 계정으로도 동일 실패
→ RBAC 조작이 불가능한 구독으로 판단하고 Azure Prowler를 범위에서 제외
- 추적** - 재검증에서 같은 조회를 az rest(raw ARM REST API)로 실행
→ 정상 작동, 구독에는 역할 할당 9개가 이미 존재
→ 구독이 아니라 az CLI의 role assignment 상위 명령어가 이 환경에서 깨진 것
- 조치** - raw REST(PUT)로 prowler-sp에 구독 Reader 부여 → Prowler Azure 완주
OCSF 375KB를 S3 업로드 → ingest → normalize → RDS 전 구간 관통
(Azure finding 확보)

정규화 파서 크래시

- 증상** - Inspector finding이 콘솔에 0건
- 추적** - EventBridge 중단을 의심
→ list-rules로 EventBridge 규칙 5개 전부 ENABLED 확인
→ Security Hub를 GeneratorId로 직접 조회하니 이미 273건이 relay
→ 파이프가 아니라 파서를 의심, 실 데이터 구조로 재현
→ Inspector의 ASFF는 ProductFields · Compliance가 명시적 null이라
raw.get(key, {})가 None을 막지 못해 배치 전체가 실패
- 조치** - None-safe 처리 + 정보 없는 제품의 기본 status를 remediated → open 교정
→ 실 finding 30건을 SQS로 흘려 검증, 같은 CVE-2024-3094를
Trivy · Inspector가 독립 탐지해 1건으로 병합 (크로스스캐너 dedup 실증)

“설계 → 목업 → 구현 → 실제 전환 → 검증” 단계로 프로젝트 진행 완료

기획 단계에서 정의한 **문제**를 설계한 **해결책**으로 모두 해결하고, **AWS·Azure** 라이브 환경에서 실 검증까지 완료

통합 → CNAPP

흩어진 보안 신호를 단일 콘솔·단일 기준으로 통합

6개 스캐너(Prowler·kube-bench·Trivy·Access Analyzer·Macie·Inspector)를 정규화해,
실 스캔 finding **432건** 통합 (**open 32 · 준수 63 · 노이즈 억제 337**, 2026-07-28 기준)

여러 스캐너·서비스를 오가며 수동으로 취합하는 과정 없이,
단일 화면·단일 우선순위 기준으로 바로 판단

상관 → Attack Path

개별로는 안 보이던 위험을 하나의 공격 체인으로 표시

라이브 환경에 3개 attack-path가 실제로 존재하며,
공개 S3 버킷 조치를 승인하자 관련 경로가 **실시간으로 소멸**(3 → 1)되는 것 확인
개별 심각도로만 보면 **놓쳤을 위험**이, **체인으로 묶이는** 순간
위험도가 격상돼 우선순위에서 빠지지 않음

경계 횡단 → Cross-Cloud

단일 클라우드 도구가 보지 못하는 멀티 클라우드 경계 횡단 공격까지 추적

실제 Azure 테넌트를 Prowler로 스캔해 확보한 실 finding **3건**을
AWS에 남은 평문 Azure 자격증명과 연결해, 5개 노드로 구성된 attack-path 그래프에서
AWS → Azure로 넘어가는 지점 1곳을 명확히 구분해 표시
AWS 전용·Azure 전용 도구 각각으로는 끊겨 보였을 공격의 흐름을, **처음부터 끝까지 추적**

자동화 → Agentic AI

요약만 하는 챗봇을 넘어, 스스로 조사해 판정까지 내리는 AI로 실현

실 스캔에서 open 상태인 finding **33건** 중, Triage 게이트를 통과한 **24건** 전부를
Bedrock이 read-only API로 스스로 조사 완료(**미조사 0건**)
관제 콘솔에 직접 로그인해 API를 하나씩 확인하지 않아도,
AI가 스스로 증거를 모아 확증까지 도달

AWS·Azure 라이브 환경에서 완전히 작동하는 CNAPP 플랫폼을 완성함

데모 영상 (1분 30초): youtu.be/Y_aPADbf3t94

감사합니다.

Contact

김진우

P. 010-3470-1503

M. kkjjww1503@naver.com

Contact

이준형

P. 010-3390-4302

M. bljh5220@gmail.com